

SCARA Two-Axis Arm

Video link	3
Team Member Work Distribution	3
System Level Block Diagram	4
Top Level Architecture	5
Block 1 Design Details	6
Block 1 Gabriel Boyd, Mechanical	6
Video link	6
Block Level Interface Validation Table	10
Interface Validation	10
Block 2 Design Details	11
Block 2 Alex Prosnik, Power Supply	11
Block Level Interface Validation Table	12
Interface Validation	14
Block 3 Design Details	15
Block 3 Alex Prosnik, Microcontroller	15
Block Level Interface Validation Table	17
Block 4 Design Details	18
1) Initialize Parameters	20
2) Define Helper Functions	20
3) G-code File Creation (Optional User Input)	20
4) G-code Processing	20
5) Initial Setup	20
6) Process G-code Commands	20
7) Finalize	20
Block Level Interface Validation Table	21
Interface Verification	22
Block 5 Design Details	23
Block 5 Dylan Sutardjo, MG996R Servo	23
Block Level Interface Validation Table	25
Interface Validation	27
Artifacts	29
Block 6 Design Details	30
Block 6 Dylan Sutardjo, PCA9685 Driver Board	30
Block Level Interface Validation Table	32
Interface Validation	34
System Level Interface Validation Table	35
Engineering Requirements	37
Verification Process	37

Gabriel Boyd 934-428-158, Dylan Sutardjo 934-434-234, Alexander Prosnik 934-431-515
Team #3

Life Cycle Assessment	39
Artifacts	40
References	43

Gabriel Boyd 934-428-158, Dylan Sutardjo 934-434-234, Alexander Prosnik 934-431-515
Team #3

Video link

<https://drive.google.com/file/d/1ETu2b9mJXGcpp6mzlq-RDytQs8gTvDUZ/view?usp=sharing>

Team Member Work Distribution

Name	Contributions	Hours worked (total)
Gabriel Boyd 934-428-158	Component research, mechanical design and manufacture, mechanical design revisions, Inverse kinematics research and Python Implementation, Servo Calibration, Arduino/C++ ESP-32 code.	60 Hours
Dylan Sutardjo 934-434-234	Servo and servo driver block, extended research on various topics, documentation, calibration, showcase site, and coding.	50 Hours
Alex Prosnik 934-431-515	Schematic design, PCB design, power supply, and coding.	45 Hours

Got feedback on this document? [Email us!](#)

System Level Block Diagram

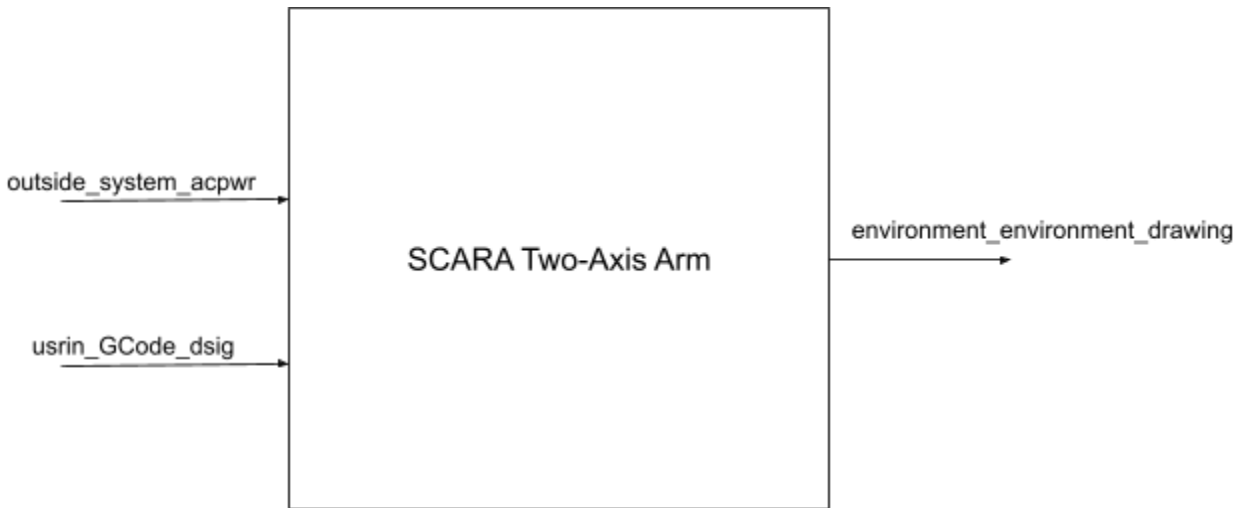


Fig #1. Black box diagram.

System Description

Our system is a SCARA two-axis robotic arm used for drawing. Different attachments could be attached to the end of the arm for different uses. It is capable of drawing a straight, 10-inch line with a rate of 4 inches per second. The arm will use commonly known interfaces and will be compatible with G-code commands, enabling it to follow specific drawing instructions. Another engineering requirement of the design is to ensure it is affordable, this means that the total cost should remain below \$200. One additional engineering requirement is the ability to quickly swap out the writing utensil at the end of the arm. This feature allows for flexibility in switching between different pens, markers, or other drawing tools, enhancing its versatility in various drawing scenarios. The system is intended to provide a reliable, easy-to-use solution for precise and consistent drawing tasks.

System Design Details and Validation

Top Level Architecture

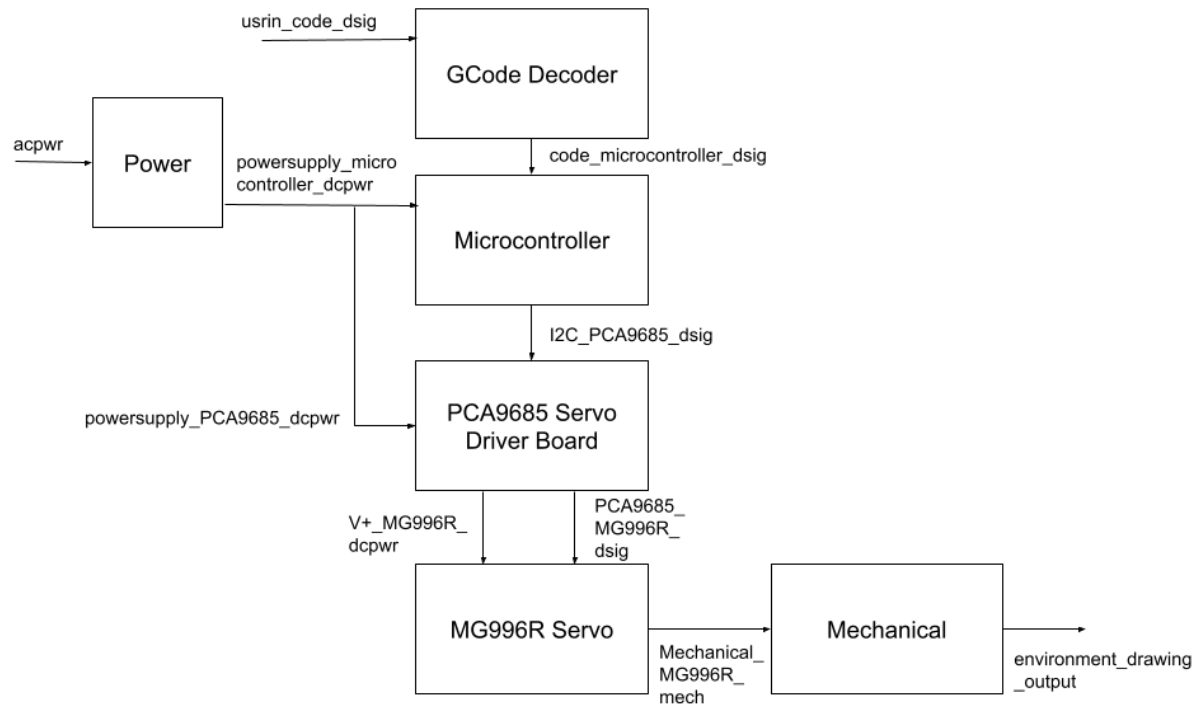


Figure #2: Top-Level Block Diagram

Block 1 Design Details

Block 1 Gabriel Boyd, Mechanical

Video link

<https://drive.google.com/file/d/1a2xmyxpmd5rb7Px6Lj30f-Y4MMWOP4Ps/view?usp=sharing>

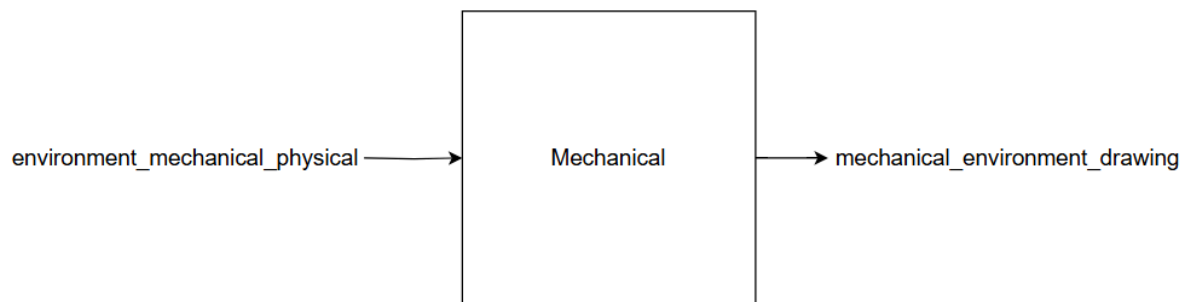


Figure #3: Mechanical Block diagram

Description

The mechanical block is responsible for physically connecting all parts of the system constituting the structure of our robot and frame. The physical interface of our mechanical system represents the physical dimensions of our frame and arms as well as the material we are using. The output of this block will be controlled arm movement, allowing us to draw, as well as supporting all other blocks and materials. This manufacturing of this block will primarily consist of 3d printing, integrating 2 roller bearings for reduced friction.

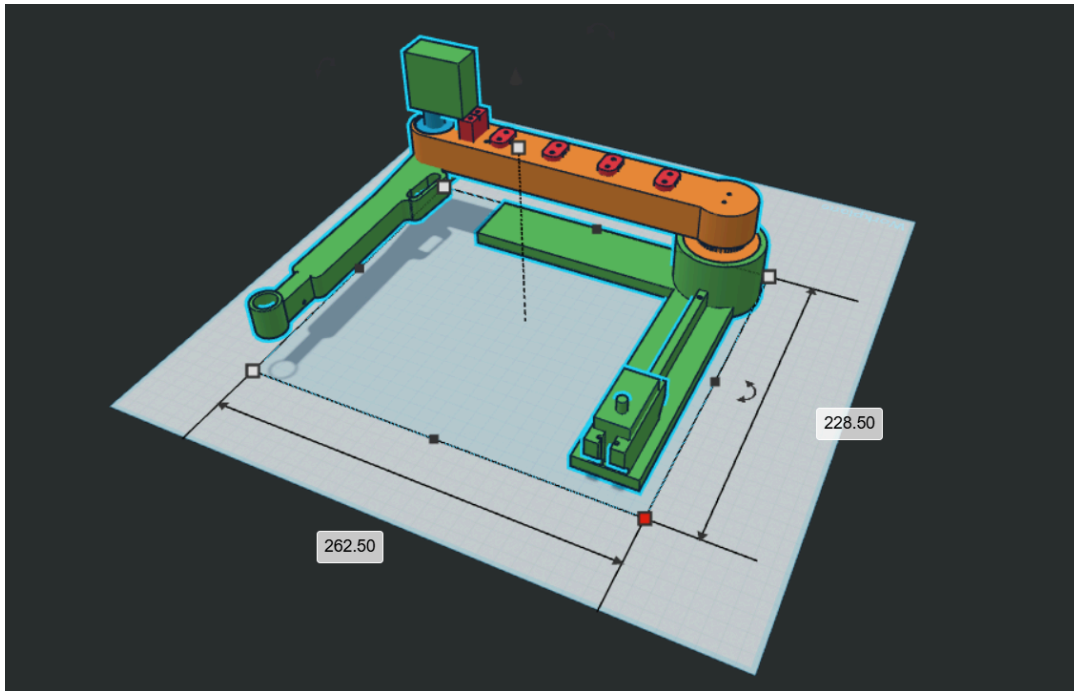


Figure #4: Mechanical Schematic

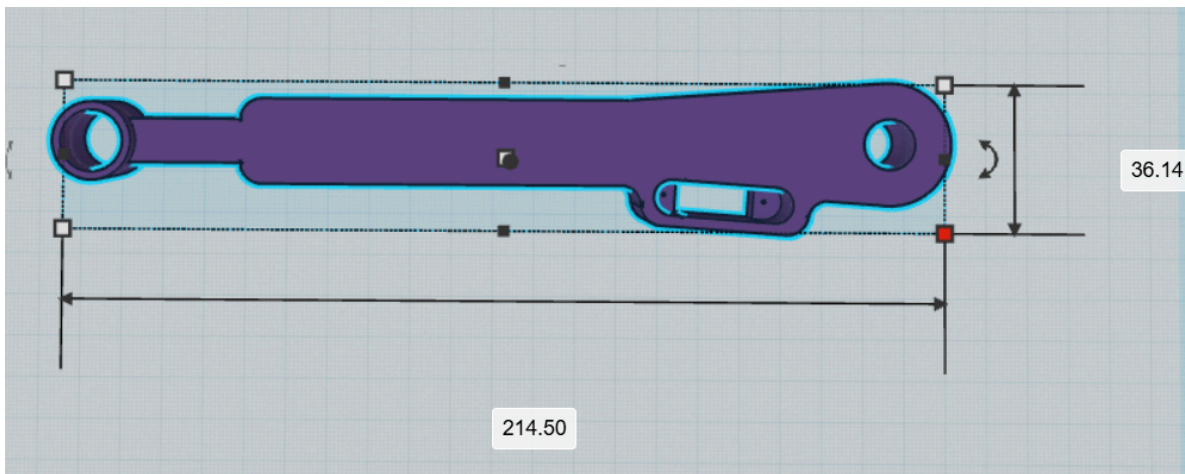


Figure #5: Elbow Arm

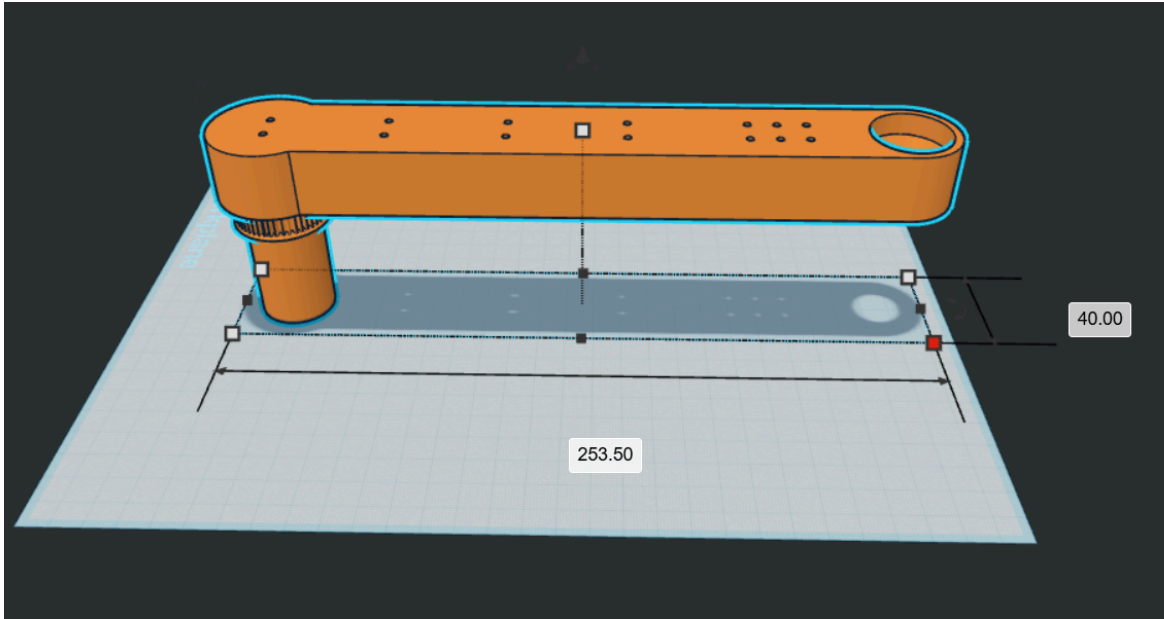


Figure #6: Shoulder Arm

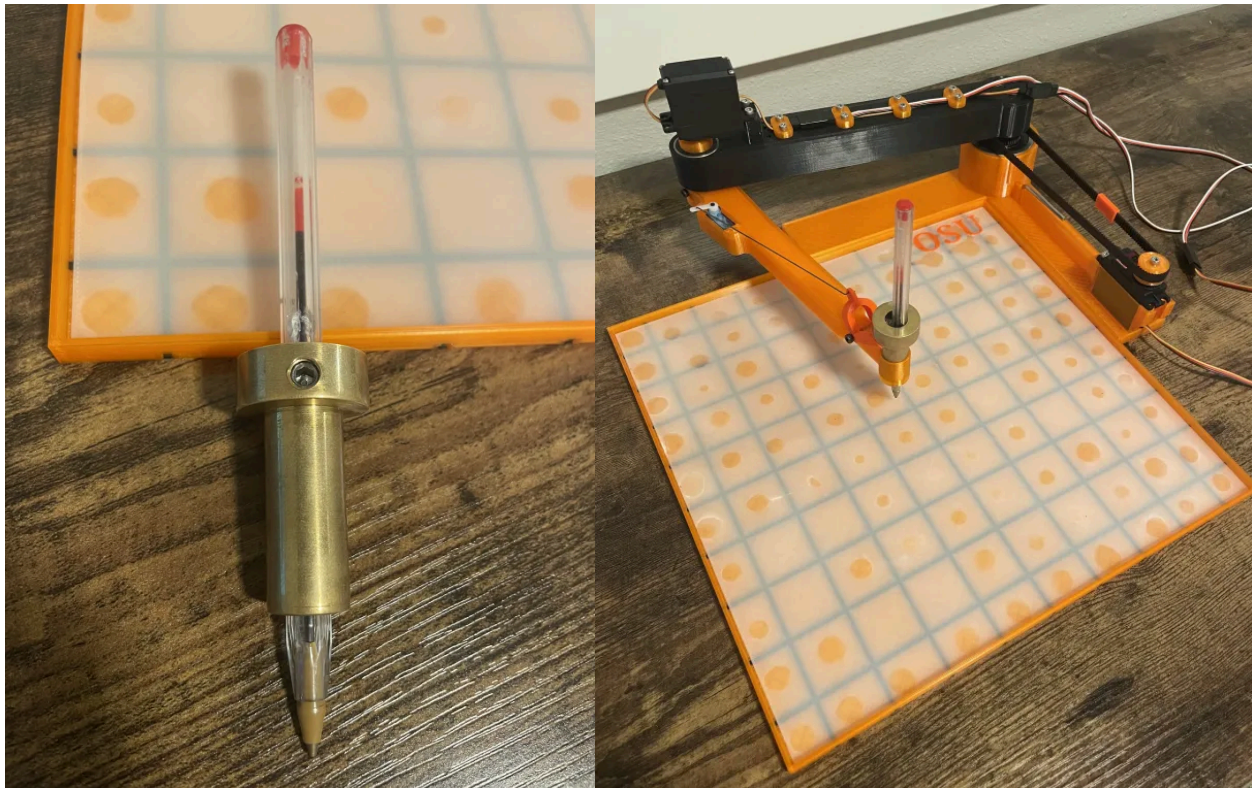


Figure #7: Physical Implementation

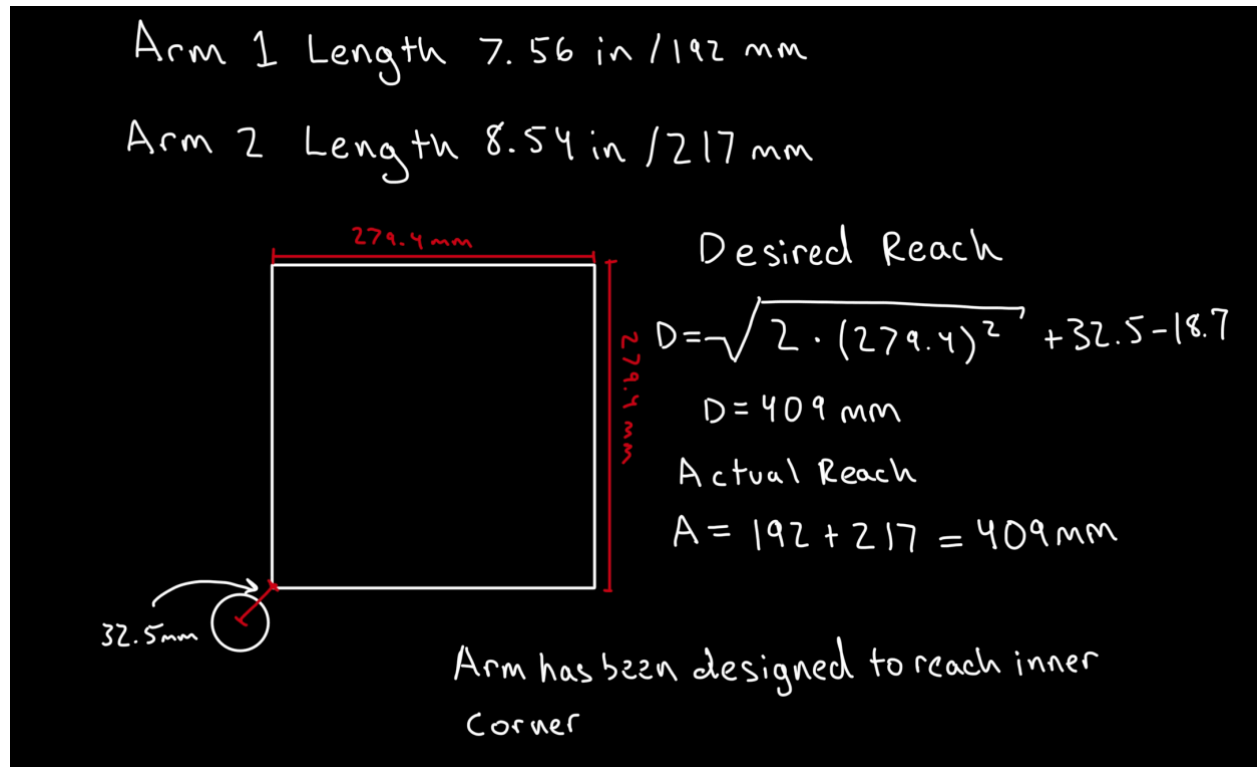


Figure #8 - Arm Dimension Validation

Block Level Interface Validation Table

mechanical_environment_drawing

Minimum Reach: 10*10in	The mechanism must be able to reach across a minimum surface size 10*10 to draw in all directions given a stationary inner joint.	The design of the arms will ensure that it can reach all parts of the effective drawing area. With two 9 inch arms this will be achieved (Reference figure 2 for calculations).
Minimum travel rate: 10in/1.5seconds	The system must be able to draw a 10 inch line in less than 2.5 seconds, meaning the mechanical block must be able to travel at a speed faster than this without damage. This provides a 1 second margin to our engineering requirement.	Roller bearings will be used ensuring that friction in the joints is minimized and the arms can travel at a rate faster than this.

environment_mechanical_physical

Inner Arm Dimension: 9 inches	This value allows us to reach all parts of our paper ensuring we can meet the engineering requirement of a 10 inch line in all directions.	Reference Figure 2 calculations for validation of dimensions.
Outer Arm Dimension: 7.75 inches	This value allows us to reach all parts of our paper ensuring we can meet the engineering requirement of a 10 inch line in all directions.	Reference Figure 2 calculations for validation of dimensions.
Minimum Outer Arm Rotation: 165 degrees	A joint movement of 160 degrees on the outer arm allows full reach of the effective drawing area (10.5*10.5in). We have a 5 degree margin.	The joints will be designed ensuring a minimum of 165 degree range of motion minimum. Reference figure 2 for validation.
Minimum Inner Arm Rotation: 100 degrees	A joint movement of 90 degrees on the inner arm	The joints will be designed ensuring a minimum of 100

	allows full reach of the effective drawing area (10.5*10.5in). We have a 10 degree margin.	degree range of motion minimum. Reference figure 2 for validation.
--	--	--

Block 2 Design Details

Block 2 Alex Prosnik, Power Supply

Description

The Power Supply block is the block responsible for providing a regulated 5 V DC power to the ESP32 microcontroller, the PCA9685 servo driver, and the 3 servo motors. This block consists of a 5 V 3 A AC-to-DC power adapter, which converts standard AC mains power (100 V-240 V) into DC 5 V output. The 5 V output is then distributed to the system. The ESP32 and PCA9685 require a stable 5 V supply, while the servo motors draw high peak currents during movement, which utilizes the 3 A capacity of the power supply.

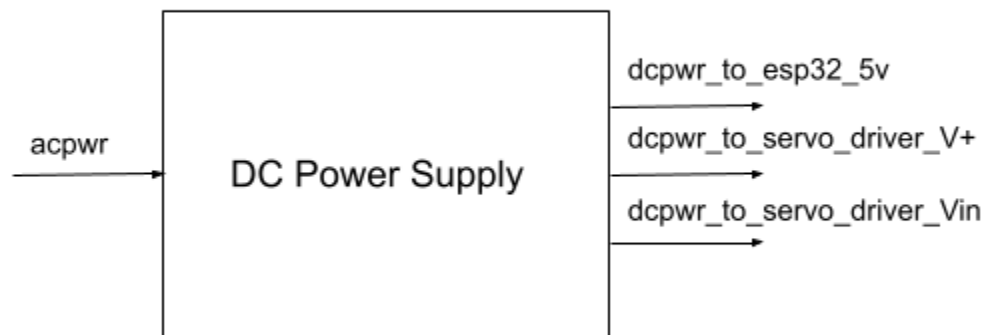


Figure #9: Power Supply block diagram

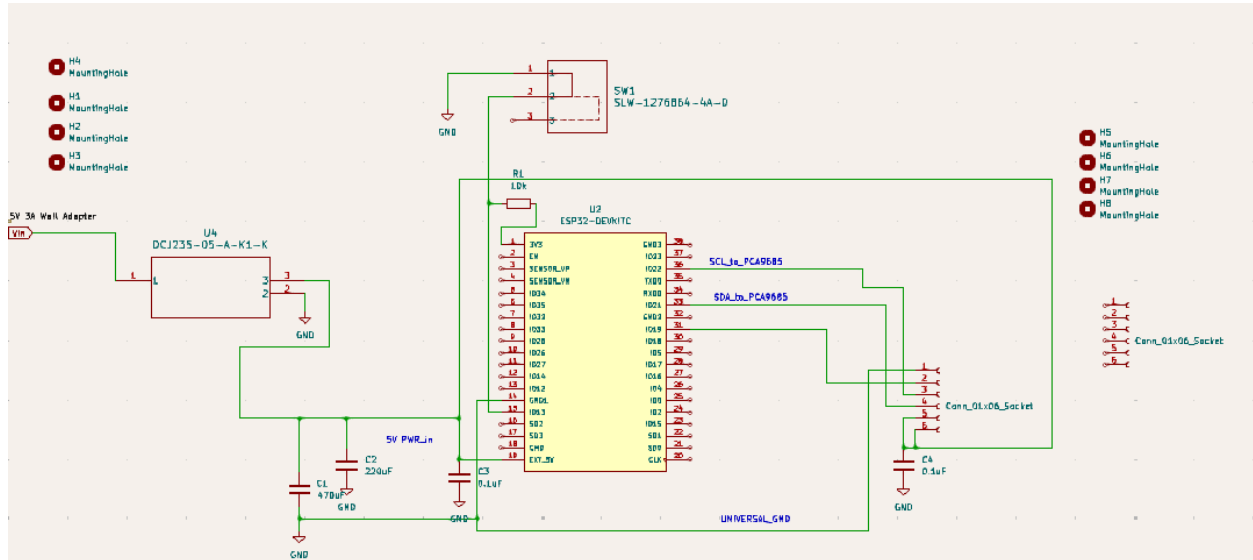


Figure #10: Power Supply Schematic

Block Level Interface Validation Table

Interface Property	Why is this interface this value?	Why do you know that your design details <u>for this block</u> above meet or exceed each property?
acpwr_dcpwr_powersupply: Input		
Vnominal: 120VAC	Standard nominal voltage for AC mains supply in North America.	The power adapter converts this voltage to 5V DC.
Other: NEMA 5-15R	Standard North American AC power plug type.	Ensures compatibility with standard wall outlets.
Inominal: 500mA	Estimated average current drawn during operation.	Based on ESP32 power consumption and servo requirements.
Ipeak: 3A	Accounts for transient inrush current when servos are active.	Ensures the power supply can handle momentary current surges.

Powersupply_esp32_dcpwr: Output

Vmin: 3.6 V	Minimum voltage for ESP32 operation.	Typical Voltage expected for the ESP32 operation. Verified from ESP32 datasheet.
Vmax: 5.25 V	Maximum recommended input voltage.	Verified using external DC power source set to 5.25 V. The board was able to function with a source of 5.25 V.
Inominal: 80 mA	Average current drawn during operation.	Verified by page 7 on the datasheet where it states the average operating current is 80 mA.
Ipeak: 500 mA	Peak current drawn during Wi-Fi communication.	Verified by page 7 on the datasheet where the maximum current is delivered from the onboard power supply is 500 mA.

Powersupply_pca9685_dcpwr: Output

Vmin: 4.8 V	The datasheet lists the typical voltage to be 5 V on pg.38 [6]	Verified using external DC power source set to 5 V. The board was able to function with a source of 5 V.
Vmax: 6 V	The datasheet lists the maximum voltage to be 6 V on pg.38 [6].	This property was verified using an external DC power source The board was able to function with a source of 6 V.
I_nominal: 40 mA	The operating mode with no load lists the typical current as 40 mA [6].	Using an electronic load. We can verify that the board is able to handle 350 mA draw.
I_peak: 2.35 A	Based on the datasheet of the MG 996R servo, the maximum current drawn at 6 V has a value of 900 mA [1]. And based on the datasheet of the GH-S37D mini-servo, the maximum current drawn is 550 mA.	Page 2 of the datasheet lists the running current range as 500 mA to 900 mA [1]. This value would be read if the voltage pushed through the servo was at least 6 V.

Verification Process

Setup:

1. Connect Vout (5V_PWR_in) and Ground (GND) to the electronic load.
2. Connect the Input terminals (Vin and GND) to a DC power supply.
3. Turn on the power supply and electronic load, but do not enable either device.

Validation of Power Supply:

1. Set the DC power supply to 5V and enable it.
2. Ensure the electronic load is not enabled, or set to a current of 0mA.
3. Observe that the following hold concurrently for at least 30 seconds:
 1. Voltage reported by the electronic load remains within the range of V_{min} to V_{max} (4.75V – 5.25V).
 2. Current through the input as reported by the power supply remains within tolerance.
4. Set the electronic load to draw 500mA and enable it.
5. Observe that the following hold concurrently for at least 30 seconds:
 1. Voltage reported by the electronic load remains within the range of V_{min} to V_{max} .
 2. Current through the input as reported by the power supply remains within tolerance.
6. Set the electronic load to draw 1A.
7. Observe that the following hold concurrently for at least 10 seconds:
 1. Voltage reported by the electronic load remains within the range of V_{min} to V_{max} .
 2. Current through the input as reported by the power supply remains within tolerance.
8. Set the electronic load to draw 3A (peak system load with all servos active).
9. Observe that the following hold concurrently for at least 3 seconds:
 1. Voltage reported by the electronic load remains within the range of V_{min} to V_{max} .
 2. Power supply does not shut down or overheat.

Block 3 Design Details

Block 3 Alex Prosnik, Microcontroller

Description

The ESP32 microcontroller block is the block responsible for processing the input commands (G-code and presets) and creating PWM signals for the servos controlling the arm. It acts as the CPU for the system, handling the communication with the PCA9685 servo driver, and managing the logic for the arm.

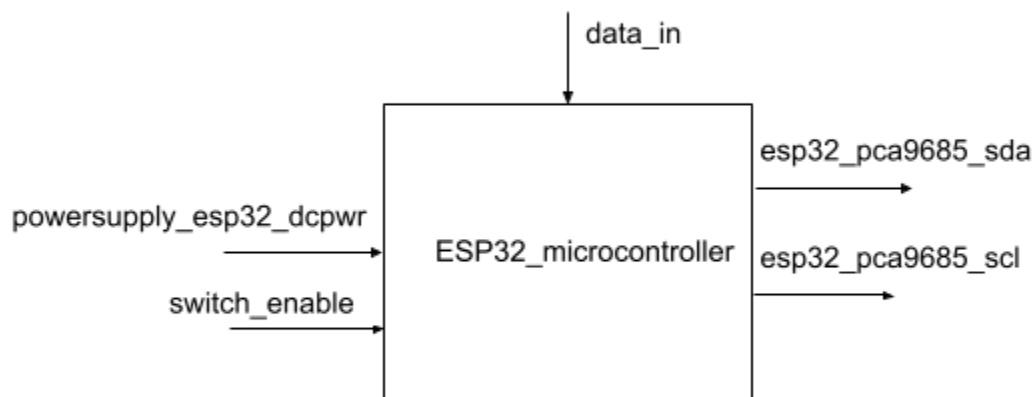


Figure #11: Microcontroller block diagram.

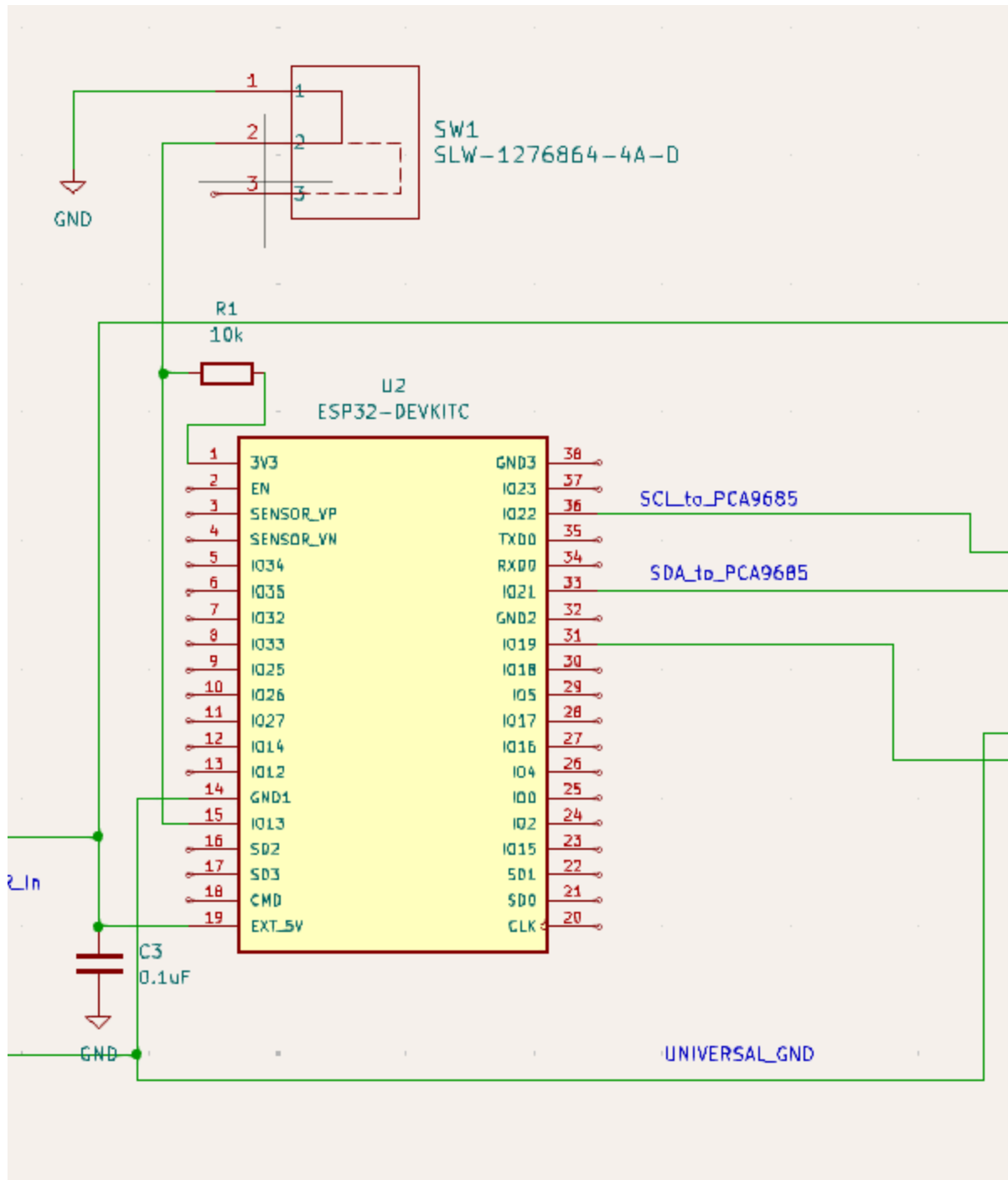


Figure #12: Schematic of ESP32-DEVKIT.

Block Level Interface Validation Table

Interface Property	Why is this interface this value?	Why do you know that your design details <u>for this block</u> above meet or exceed each property?
--------------------	-----------------------------------	--

ESP32_microcontroller: Input

G-code Commands	Commands received via Wi-Fi to control movements and drawing patterns.	ESP32 supports Wi-Fi communication for data transmission. Specified in Wi-Fi Features (page 9).
DC power Vmin: 3.6 V	Minimum voltage for ESP32 operation.	Verified with Analysis and Optimization of DC Supply Range for the ESP32 Development Board. By Joshua Kim
Vmax: 5.25 V	Maximum recommended input voltage.	Verified with Analysis and Optimization of DC Supply Range for the ESP32 Development Board. By Joshua Kim.
Inominal: 80 mA	Average current drawn during operation.	Verified by page 7 on the datasheet where it states the average operating current is 80 mA.
Ipeak: 500 mA	Peak current drawn during Wi-Fi communication.	Verified by page 7 on the datasheet where the maximum current is delivered from the onboard power supply is 500 mA.
Switch	The switch functions as the emergency stop for the system.	Verified by testing switch behavior during activation.

ESP32_microcontroller: Output

3 PWM Signals	Sends 3 PWM signals to PCA9685 servo driver to	ESP32 GPIOs generate PWM signals.
---------------	--	-----------------------------------

	control arm, base, and pen servo positions.	
50 Hz PWM Frequency	50 Hz for servo control.	Standard frequency for servo motors.
Data_in I2C Communication	Wires are connected to SDA and SCL.	I2C protocol supported by ESP32 hardware.

Block 4 Design Details

Block 4 Gabriel Boyd's **BLOCK 2**, Code

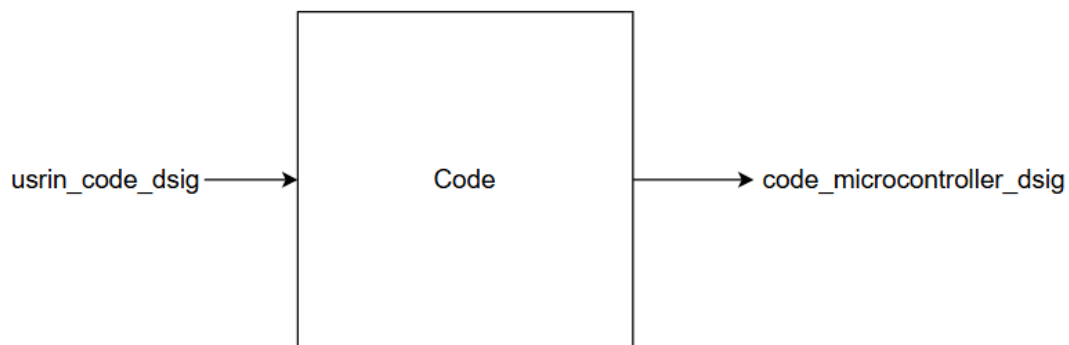


Figure #13: Code Block Diagram

Description

The code block diagram is responsible for taking G-code user inputs and transforming them into usable degree and speed data outputs for the microcontroller. This block maps X-Y coordinates and speeds into angles and delays for the servos to physically replicate G-code commands. The chosen platform for this task is Python where 2 angles, 1 delay, and 1 enable will be formatted and written to a text file.

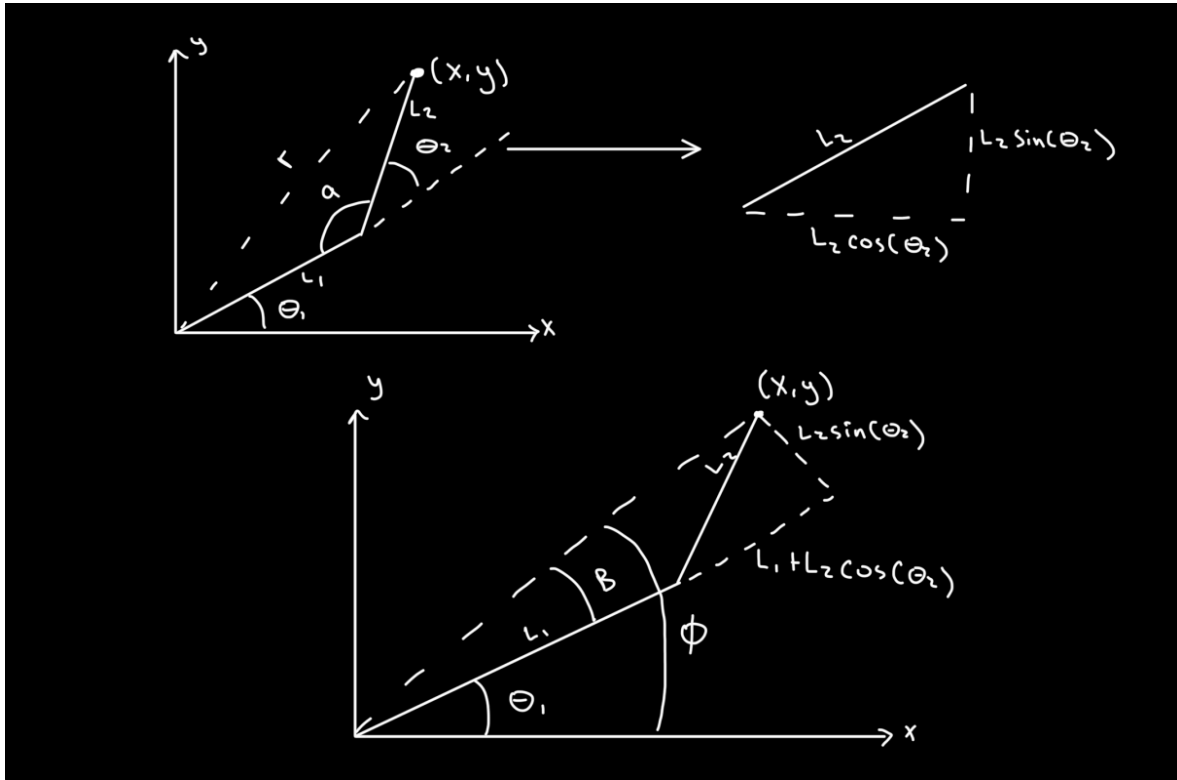


Figure #14: Inverse Kinematics Solution Diagrams[1*]

Inverse Kinematics Derivation

$$r = \sqrt{x^2 + y^2}$$

Using the Law of Cosines, we can write:

$$r^2 = L_1^2 + L_2^2 - 2L_1L_2 \cos(180 - \theta_2)$$

$$\cos(180 - \theta_2) = -\cos(\theta_2)$$

$$\theta_2 = \arccos\left(\frac{r^2 - L_1^2 - L_2^2}{2L_1L_2}\right)$$

$$B = \arctan 2\left(\frac{L_1 \sin(\theta_1)}{L_1 + L_2 \cos(\theta_2)}\right)$$

$$\theta_1 = \phi - B = \arctan 2(y, x) - \arctan 2\left(\frac{L_2 \sin(\theta_2)}{L_1 + L_2 \cos(\theta_2)}\right)$$

Pseudocode

1) Initialize Parameters

- Define physical parameters (link lengths, scaling factors, offsets)
- Define points per cm
- Define physical workspace boundaries
- Calculate centered workspace dimensions

2) Define Helper Functions

- Function to calculate angles from coordinates
- Function to scale shoulder angle
- Function to calculate delays based on distance, speed, and points per cm

3) G-code File Creation (Optional User Input)

- Ask user if they want to create a new G-code file
- If yes:
 - Open output file
 - Display instructions to user
 - Loop to accept commands until user signals complete
 - Validate each command
 - Check for valid coordinates within boundaries
 - Check for valid feed rates
 - Write valid commands to file
 - Write program end command
 - Close file

4) G-code Processing

- Open and read the G-code file
- Open output file for angle/delay commands

5) Initial Setup

- Calculate center position angles
- Find first movement position
- Initialize positioning mode and unit settings
- Calculate path from center to first position
- Write initial movement commands to output file

6) Process G-code Commands

- For each G-code line:
 - Skip comments and empty lines
 - Handle positioning mode commands (absolute/relative)
 - Handle unit commands (inches/millimeters)
 - Handle tool change commands
 - Process movement commands:
 - Extract coordinates and feed rate
 - Convert units if needed
 - Apply absolute/relative positioning
 - Map coordinates to robot space
 - Calculate points along path based on density setting
 - For each point:
 - Calculate angles
 - Calculate appropriate delay
 - Write command to output file
 - When end program command is found:
 - Write final pause command
 - Calculate path back to center
 - Write return path commands
 - Write final position command

7) Finalize

- Close output file
- Display completion message with line count

THETA1	THETA2	DELAY	ENABLE
72.23	111.85	99	1
71.90	111.24	99	1
71.58	110.62	99	1
71.29	109.99	99	1

Figure #15: Example File Output

Block Level Interface Validation Table

usrin_code_dsigs

(x,y) G-code coordinate range : (0 : 10.25 , 0 : 10.25) inches	This range allows for full reach of the available coordinate plane, currently shown in inches. This can be easily adjusted to accept a wide variety of projects with different units by simply changing the scale.	Python will ensure all G-Code inputs fall within the defined range, an error message will be thrown if this is not true.
G-code speed range: 1-120 mm/s	This range allows us to meet the system requirement of a 10 inch line in 2.5 seconds (102mm/s) with a margin of 18mm/s	Python will provide an error if speed inputs are outside of this range, ensuring all inputs are within this range.

code_microcontroller_dsigs

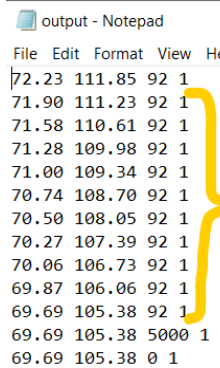
Points per cm (PPcm): 4 PPcm/10.16PPin	Our servos have an accuracy of 0.8 degrees [1], this ensures we will have minimal accuracy loss in our PWM signal as it will be significantly more accurate than our margin of error of 0.25in, 4PPcm should provide a margin of error < 0.1cm.	Python encodes the file output using a distance formula (outputting cm), followed by multiplying by PPcm to obtain total points. This allows us to obtain our accuracy of 4 points per cm.
Max command lines output 50,000	To ensure accurate G-code replication by the servos the system must store an adequate	There is a maximum number of 50,000 lines output from Python. If the program exceeds this

	number of commands. 50,000 lines will provide plenty of accuracy for our G-code while still fitting on our ESP-32 flash. Each line is 16 Bytes (2 float, 1 integer, 1 boolean, 3 bytes for formatting). Given we have 4MB of flash [2*], if we reserve 1.5MB for our file we have a maximum number of 93,000 lines, we have a 43,000 line margin.	maximum Python will provide an error message informing the user their request exceeds the maximum number of lines.
--	---	--

Verification Process

All verification will be done in Python terminal

1. (x,y) G-code coordinate range
 - a. Begin by inputting G-code inside acceptable coordinate range (between (0 : 10.25, 0 : 10.25)).
 - b. Open the file outputted by Python and verify that the coordinates and delays were encoded.
 - c. Input G-code outside of the acceptable coordinate range (Ex $x > 10.25$ or $x < 0$).
 - d. Verify that Python provided an error and did not compile invalid command into the G-Code file.
2. G-code speed range
 - a. Begin by inputting G-code inside acceptable speed range (1-120mm/s).
 - b. Open the file outputted by Python and verify that the coordinates and delays were encoded.
 - c. Input G-code outside of the acceptable speed range ($\text{Speed} < 1$: $\text{Speed} > 120$ mm/s).
 - d. Verify that Python provided an error and did not compile angles and delays into
3. Points Per cm
 - a. Begin by inputting G-code
 - i. G20
 - ii. G0 X6.125 Y5.125.
 - b. Verify that the output is 10 lines (we traveled 1 inch from the starting location so we should have 10 points)



output - Notepad

File	Edit	Format	View	Help
72.23	111.85	92	1	
71.90	111.23	92	1	
71.58	110.61	92	1	
71.28	109.98	92	1	
71.00	109.34	92	1	
70.74	108.70	92	1	
70.50	108.05	92	1	
70.27	107.39	92	1	
70.06	106.73	92	1	
69.87	106.06	92	1	
69.69	105.38	92	1	
69.69	105.38	5000	1	
69.69	105.38	0	1	

4. Max command lines output
 - a. Input G-code which results in 500<50,000 lines encoded in the text file (Python will output the number of lines encoded), ensuring G-code is within acceptable coordinate and speed range.
 - b. Open the file outputted by Python and verify that the coordinates and delays were encoded.
 - c. Input G-code which results in > 50,000 lines encoded in the text file (Python will output the number of lines encoded).
 - d. Verify Python provided an error claiming the resulting file would have been greater than 50,000 lines.

Block 5 Design Details

Block 5 Dylan Sutardjo, MG996R Servo

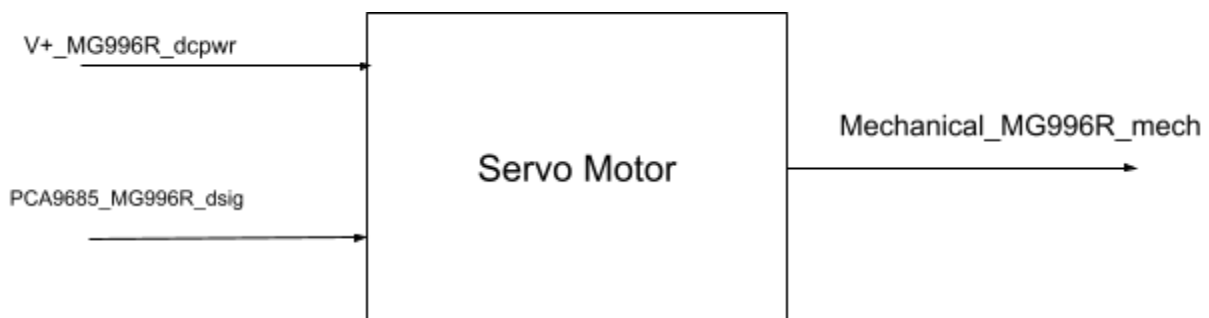


Figure #16: Block diagram of MG996R servo

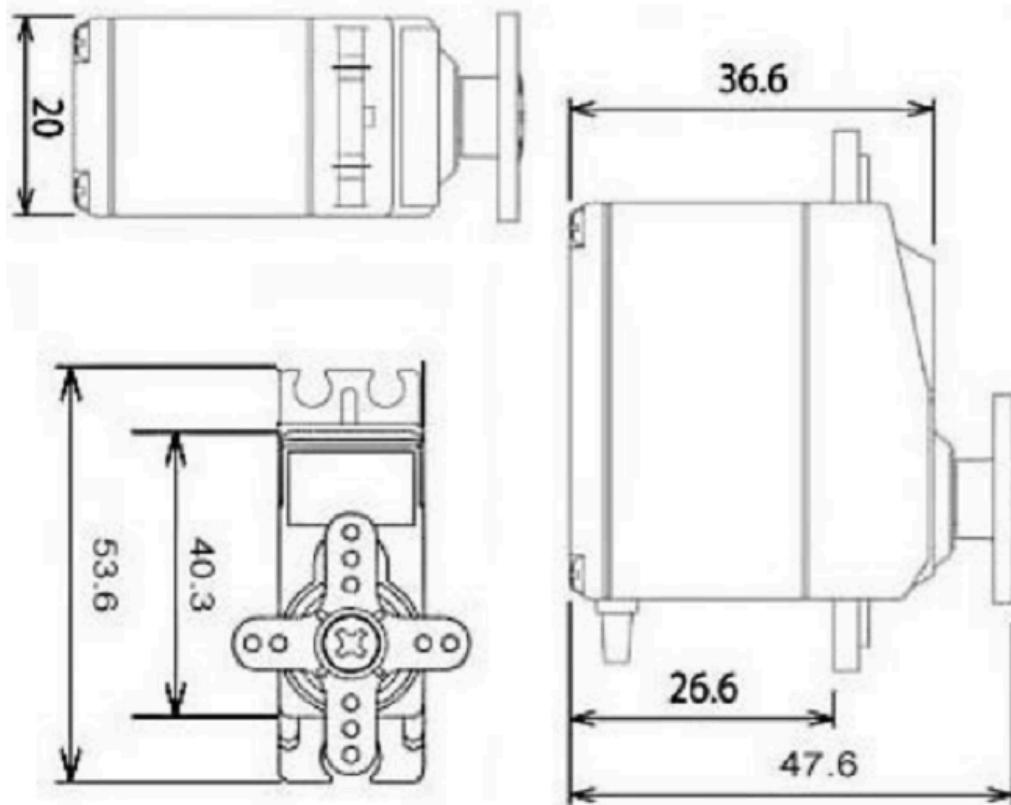


Figure #17: Exterior schematic of MG996R servo with measurements [1]

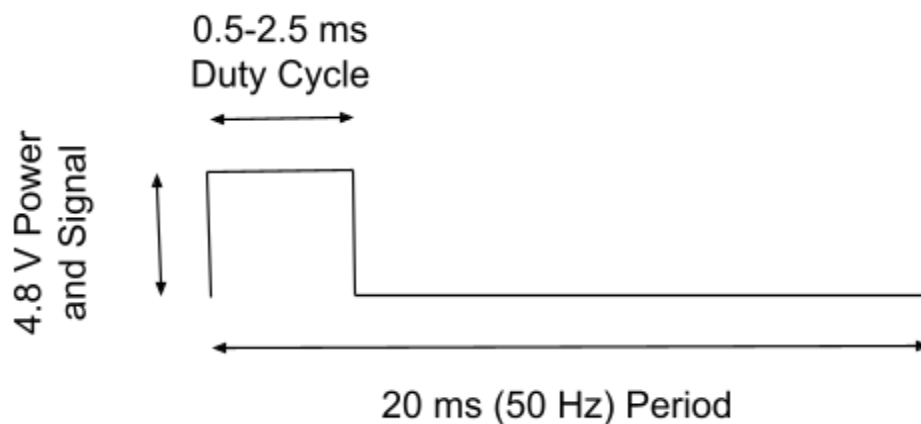


Figure #18: PWM Signal and duty cycle [1]

Description

The system will contain two MG996R High Torque Metal Gear Dual Ball Bearing 55g servos. They will be in charge of the accurate movement of the arm. The MG996R servo has metal gearing, allowing for high 10 kg stalling torque. It has an operating angle of 180 degrees and an operating speed of 0.30 s/60 degrees when powered with 5 V. The inputs for the servo will be dc

Got feedback on this document? [Email us!](#)

power 5 V +/- 0.4 V. It will also take in any PWM signal received from the servo driver. The outputs of the servo will interface with the mechanical block as mechanical_servo_rotation. One servo will rotate the entire arm, while a second is used to rotate just the forearm.

Block Level Interface Validation Table

Interface Property	Why is this interface this value?	Why do you know that your Block design details meet or exceed each property (reference block details as needed)?
--------------------	-----------------------------------	---

V+_MG996R_dcpwr: Input

Vmin: 4.6 V	The motor requires power to operate and based on the datasheet, The minimum power supply is 4.6 V. With a typical voltage of 5 V [1].	The value is within the range of operating voltage given by page 2 of the MG996R datasheet, which is 4.6 V to 7.2 V [1].
Vmax: 6 V	The motor requires power to operate and based on the datasheet, the maximum power supply is 7.2 V. With a typical voltage of 5 V [1].	The value is within the range of operating voltage given by page 2 of the MG996R datasheet, which is 4.8 V to 7.2 V [1]. To protect the components, we will not go above a voltage of 6 V.
I_nominal: 500 mA	Based on the datasheet, the typical current consumption with an unloaded output is 500 mA [1].	Page 2 of the datasheet lists the running current range as 500 mA to 900 mA [1]. This I_nominal value falls within that range for lower voltage values.
I_peak: 900 mA	Based on the datasheet, the maximum current drawn at 6 V has a value of 900 mA [1].	Page 2 of the datasheet lists the running current range as 500 mA to 900 mA [1]. This value would be read if the voltage pushed through the servo was at least 6 V.

PCA9685_MG996R_dsig: Input

PWM Frequency: 50 Hz	According to the datasheet, for servo control, the signal is sent at a frequency of 50 Hz [6].	By attaching an oscilloscope to the output pin, we are able to verify the frequency of which the signal is being sent. We can set the angle to zero degrees and validate this.
Duty Cycle: 0.5-2.5 ms	The ratio of the time the signal is "on" to the total period comes out to 2.5 ms [6].	Using oscilloscope probes, we can view the full period of the signal and find the duty cycle. Duty cycle = (Pulse width / Period) * 100

mechanical_servo_rotation: Output

Range_of_Op: 180 degrees	The angle of operation is listed as 180 degrees on the datasheet [1].	After using test code, we were able to confirm that the servo is capable of rotating 180 degrees. Using trigonometry, we are able to get an accurate measurement.
Op_Speed: 0.30 s/60 degrees +/- 0.03 s	At 4.8 V, the speed of operation is said to be 0.30 s/60 degrees [1]. This means this motor will meet the engineering requirement of 4 in. per second	By taking a video of the servo moving 60 degrees, we can use frame by frame analysis to clock the time it takes for the motor to rotate. The results matched our listed operating speed.
Max_torque: 9.4 kg/cm	The stall torque is listed at 9.4 kg/cm when V_{in} is 4.8 V [1].	Torque is $Force \cdot Distance \cdot \sin(\theta)$. We obtain the force by attaching a force scale and measuring the weight required to stall the motor. We can get the distance by measuring the length from the center of the gear to where the scale is attached. θ is the angle to which force is applied. We will apply it at 90 degrees.

Verification process

Motor Setup

1. Hook up servo driver to microcontroller
 - a. PIN 19 to OE
 - b. PIN 21 to SDA
 - c. PIN 22 to SCL
 - d. GND to GND
2. Attach a 5V source to the PCA9685 VCC pin and the V+ pin
 - a. Attach grounds
3. Attach a 5V source to the 5 microcontroller pin
 - a. Attach grounds
4. Hook up servo to channel zero on PCA9685

Validation of torque

1. Attach a hook contraption that is $\frac{1}{8}$ inch away from the center of the gear
2. Hook a weight scale in kg to the end
3. Apply code that moves the arm and pull on it using the scale, causing it to stall
4. Pull at 90 degrees
5. Read the weight measured when servo is stalled
6. Multiply that value by the distance away from the center and $\sin(90)$ to find the torque

Validation of Range

1. Flash code that rotates the arm 180 degrees

```
1  #include <Wire.h>
2  #include <Adafruit_PWM_ServoDriver.h>
3
4  // Initialize the PCA9685 module
5  Adafruit_PWM_ServoDriver pwm = Adafruit_PWM_ServoDriver();
6
7  // Define servo settings
8  #define SERVOMIN  102 // Minimum pulse length count (for 0 degrees)
9  #define SERVOMAX  512 // Maximum pulse length count (for 180 degrees)
10 #define PWM_FREQ  50  // Frequency for servo control (50Hz)
11
12 // Function to map angle to pulse length
13 int angleToPulse(int angle) {
14   return map(angle, 0, 180, SERVOMIN, SERVOMAX);
15 }
16
17 void setup() {
18   Serial.begin(115200); // Start serial communication
19   Serial.println("Initialising PCA9685 Servo Controller...");
20
21   // Initialize the PCA9685
22   pwm.begin();
23   pwm.setPWMFreq(PWM_FREQ); // Set frequency to 50 Hz
24   delay(10); // Small delay for stability
25
26   Serial.println("PCA9685 Initialized.");
27 }
28
29 void loop() {
30   for (int angle = 0; angle <= 180; angle += 1) {
31     // Map angle to pulse length and set PWM
32     int pulse = angleToPulse(angle);
33     pwm.setPWM(0, 0, pulse); // Channel 0
34     Serial.print("Servo moved to ");
35     Serial.print(angle);
36     Serial.println(" degrees.");
37
38     delay(20); // Wait for 20 ms between steps
39   }
40
41   for (int angle = 180; angle >= 0; angle -= 1) {
42     // Map angle to pulse length and set PWM
43     int pulse = angleToPulse(angle);
44     pwm.setPWM(0, 0, pulse); // Channel 0
45     Serial.print("Servo moved to ");
46     Serial.print(angle);
47     Serial.println(" degrees.");
48
49     delay(20); // Wait for 20 ms between steps
50   }
51 }
```

2. Align center of gear with edges of angles
3. Confirm that the start gear rotates the full range of motion.

Validation of Speed

1. Flash code that moves the servo 60 degrees

```
1 #include <Wire.h>
2 #include <Adafruit_PWMServoDriver.h>
3
4 // Initialize PCA9685 module with the default I2C address (0x40)
5 Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver();
6
7 // Define servo settings
8 #define SERVOMIN 150 // Minimum pulse length count (0 degrees)
9 #define SERVOMAX 600 // Maximum pulse length count (180 degrees)
10 #define SERVO_FREQ 50 // Analog servos run at ~50 Hz updates
11
12 // Function to map the angle (in degrees) to pulse length
13 int angleToPulse(int angle) {
14     return map(angle, 0, 180, SERVOMIN, SERVOMAX);
15 }
16
17 void setup() {
18     Serial.begin(115200); // Start serial communication
19     Serial.println("PCA9685 Servo Speed Test");
20
21     // Initialize the PCA9685
22     pwm.begin();
23     pwm.setPWMFreq(SERVO_FREQ); // Set the frequency to 50 Hz (standard for servos)
24     delay(500); // Wait for PCA9685 to initialize
25 }
26
27 void loop() {
28     // Move servo to 60 degrees
29     int pulse60 = angleToPulse(60); // Get the pulse width for 60 degrees
30     pwm.setPWM(0, 0, pulse60); // Set PWM on channel 0
31
32     delay(1000); // Wait for 1 second
33
34     // Move servo back to 0 degrees
35     int pulse0 = angleToPulse(0); // Get pulse width for 0 degrees
36     pwm.setPWM(0, 0, pulse0); // Move back to 0 degrees
37
38     delay(1000); // Wait for 1 second
39 }
```

2. Align center of gear with edges of angles
3. Time how long it takes for gear to move 60 degrees

Artifacts

Range of motion validation:

https://drive.google.com/file/d/11iMn-oTi_lhjVGy_zgi9MoSk3Mgyp4a2/view?usp=sharing

Speed validation:

<https://drive.google.com/file/d/1s7ISkQsPrWJqYTr9v-LafKPM4jJ4BQTf/view?usp=sharing>

Got feedback on this document? [Email us!](#)

Block 6 Design Details

Block 6 Dylan Sutardjo, PCA9685 PWM Servo Driver Board

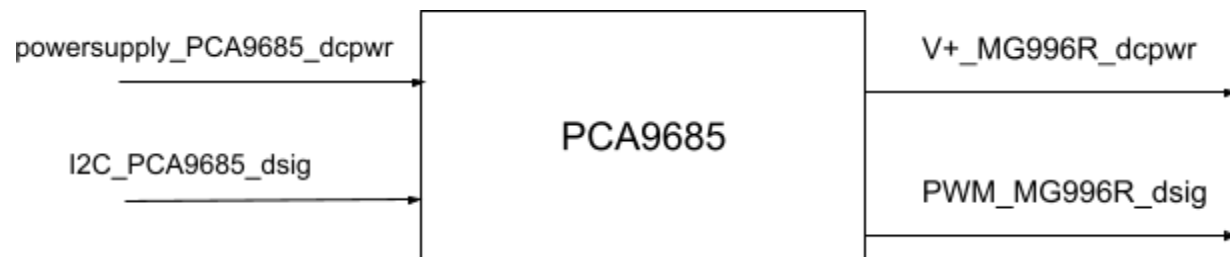


Figure #19: Block diagram of PCA9685 PWM servo driver board.

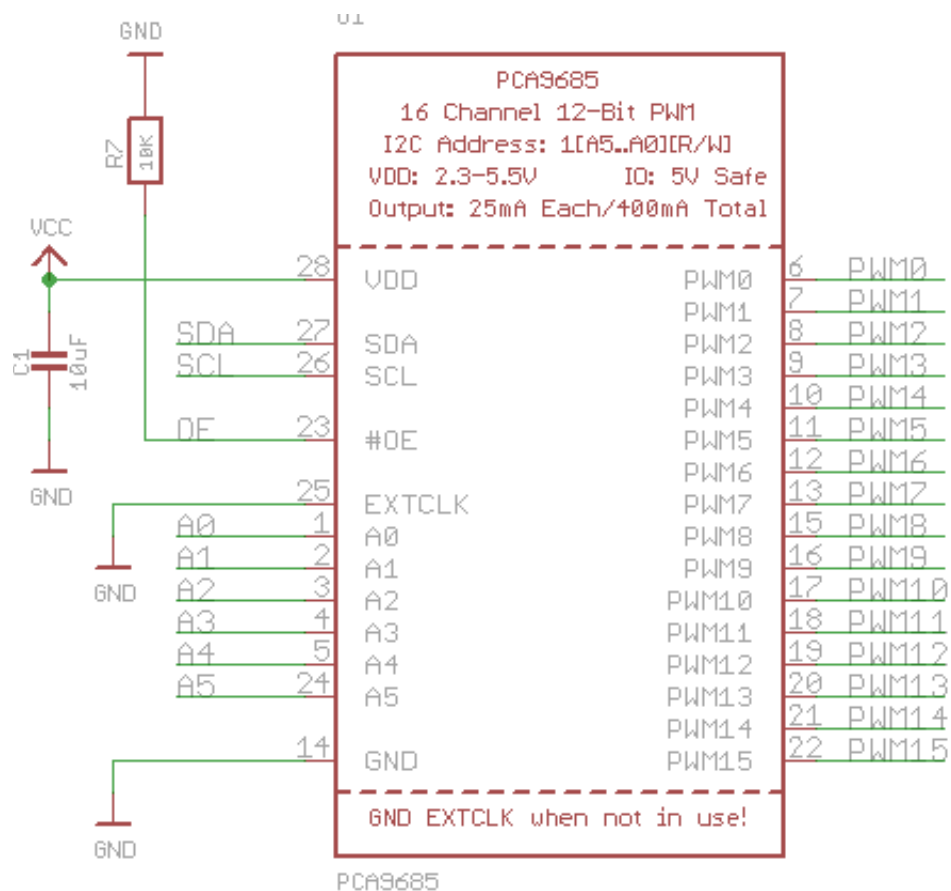


Figure #20: Schematic of PCA9685 board with output pins, input voltage management logic, and OE pin logic [7].

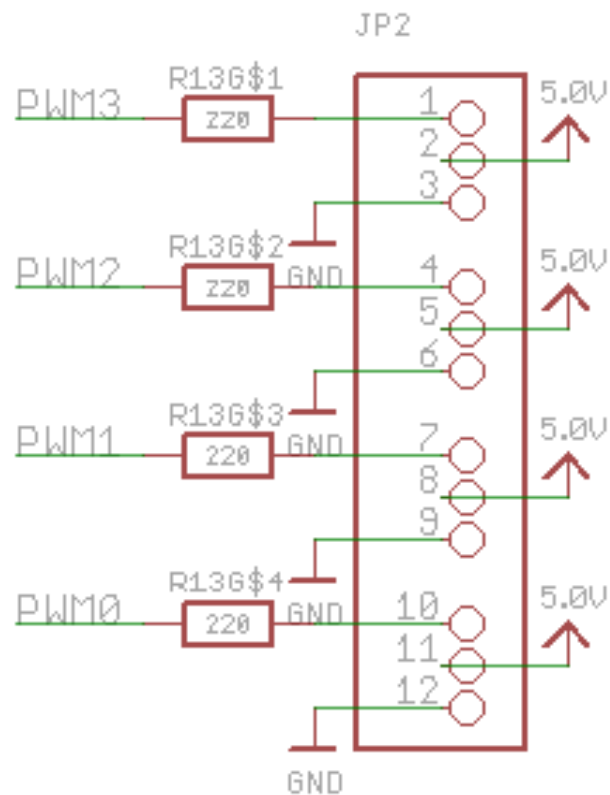


Figure #21: One resistor layout and pin layout for output pins. Corresponding to channels 0 to 3 [7].

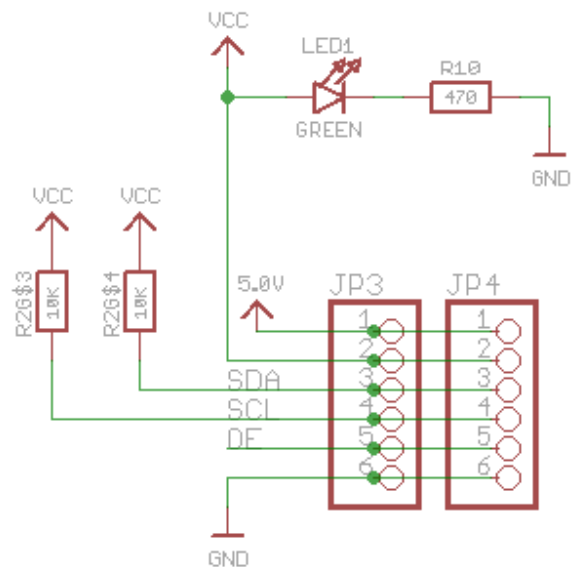


Figure #22: Input pin layout for the PCA9685 board and logic [7].

Description

This block includes the single PCA9685 PWM servo driver board. This driver powers the boards and transmits the signal from the microcontroller to the servos. For outputs, the board has 16 sets of 3 pins: PWM, V+, and GND. PWM pin is for transmitting command signals. V+ provides power for the servos, and the ground pin is a general ground. These pins will output to the MG996R servo motor. Input pins the board has include GND, VCC, and V+ for powering both the board and the motors. The board also has control pins labeled SDA, SCL, and OE. SCL is a clock pin, SDA is a data pin, and OE is output enable [7]. This means that OE can be used to quickly disable all outputs. When this pin is low, all output pins are enabled. Each PWM runs independently, but all will have the same PWM frequency [7].

Block Level Interface Validation Table

Interface Property	Why is this interface this value?	Why do you know that your Block design details meet or exceed each property (reference block details as needed)?
powersupply_PCA9685_dcpwr: Input		
Vin: 5 V	The datasheet lists the typical voltage to be 5 V on pg.38 [6].	This property was verified using an external DC power source set to 5 V. The board was able to function with a source of 5 V.
Vmax: 5.5 V	The datasheet lists the maximum voltage to be 5.5 V on pg.38 of the datasheet [6].	This property was verified using an external DC power source. The board was able to function with a source of 5.5 V.
I_nominal: 50 mA	The operating mode with no load lists the typical supply current as 50 mA on pg.38 of the datasheet [6].	Using the voltage supply, we can verify that the board typically draws around 50 mA of current at a typical voltage of 5 V.
I_peak: 400 mA	The maximum supply current for the board is listed as 400 mA [6] on pg.38 of the datasheet.	Using the voltage supply, we can verify that the board never draws a current higher than 400 mA.
I2C_PCA9685_dsig: Input		

3 PWM Signals	Sends 3 PWM signals to PCA9685 servo driver to control arm, base, and pen servo positions.	ESP32 GPIOs generate PWM signals for control of the servo. Should the servos move as intended, the board is receiving and transmitting the signal properly.
Data_in_I2C Communication	Wires are connected to SDA and SCL. These are the default I2C pins [2]. The SCL clock frequency is listed on page 40 of the PCA9685 datasheet [6].	I2C protocol supported by ESP32 hardware [2]. We are using PIN 22 which is an I2C pin.

V+_MG996R_dcpwr: Output

Vout: 4.5 V	The minimum output voltage listed in the data sheet is -0.5 V of the input [6]. Since the typical voltage is 5 V, this gives us a minimum voltage of 4.5 V.	The board allows for a range of VDD-0.5 to 5.5 V at high level voltages [6]. This gives us a lower bound of 4.5 V, which we can see the board outputting around this value.
Vmax: 5.5 V	The maximum output voltage listed in the data sheet by adafruit is 5.5 V [6].	The maximum output voltage is listed as 5.5 V [6] and the V+ pin will output a peak of this voltage.
I_nominal: 25 mA	We will be using the board at 5 V. The output current on the pin is 25 mA [6].	The board is rated to output current through V+ pin 25 mA. We can see this by measuring the output current at this pin.
I_peak: 100 mA	We will be using the board at 5 V. This means the maximum current draw is 300 mA [6]. This value is well within the safe range.	This board is rated to have a total output current of 400 mA. 300 mA is within the range of operation.

PCA9685_MG996R_dsig: Output

PWM Frequency: 50 Hz	According to the datasheet, for servo control, the signal is sent at a frequency of 50 Hz [6].	By attaching an oscilloscope to the output pin, we are able to verify the frequency of which the signal is being sent. We can set the angle to zero degrees and validate this.
----------------------	--	--

Duty Cycle: 0.5-2.5 ms	The ratio of the time the signal is "on" to the total period comes out to 2.5 ms [6].	Using oscilloscope probes, we can view the full period of the signal and find the duty cycle. Duty cycle = (Pulse width / Period) *100
------------------------	---	---

Verification process

1. Set up DC power supply to input V+ and VCC on the PCA9685 board.
 - a. Hook up grounds to designated pins
2. Connect output terminals to the electronic load.
 - a. Ensure electronic load and power supply share common ground.
3. Turn on both, but do not enable the devices.

5 V testing

1. Set DC power supply to 5 V.
2. Set electronic load to 0 mA.
3. Take note of the voltage and current being drawn on the electronic load.
4. Set DC power supply to 5 V.
5. Set electronic load to 25 mA.
6. Take note of the voltage and current being drawn on the electronic load.
7. Set DC power supply to 5 V.
8. Set electronic load to 100 mA.
9. Take note of the voltage and current being drawn on the electronic load.

5.5 V testing

1. Set DC power supply to 5.5 V.
2. Set electronic load to 0 mA.
3. Take note of the voltage and current being drawn on the electronic load.
4. Set DC power supply to 6 V.
5. Set electronic load to 25 mA.
6. Take note of the voltage and current being drawn on the electronic load.
7. Set DC power supply to 5.5 V.
8. Set electronic load to 100 mA.
9. Take note of the voltage and current being drawn on the electronic load.

Testing PWM inputs

1. Power and flash code on to the ESP32 board.
2. Attach oscilloscope ground to GND pin.

Got feedback on this document? [Email us!](#)

3. Attach the probe to GPIO 22.
4. Flash code with 50 Hz frequency and 0.5 pulse.
5. Adjust the oscilloscope so that wave and period are clearly seen.

Testing PWM and I2COutputs

1. Power the board with a DC power supply set to 5 V.
2. Hook up ESP32 to the PCA9685
 - a. PIN 19 to OE
 - b. PIN 21 to SDA
 - c. PIN 22 to SCL
 - d. GND to GND
3. Power and flash test code onto ESP32.
4. Hook up the oscilloscope to the PWM output pin of the PCA9685.
5. Observe and adjust the wave of the signal.
 - a. Measure the period using the cursors.
 - b. Measure the signal width using the cursors.
6. Duty cycle = (Pulse width / Period) * 100
7. Frequency is listed at the bottom.

System Level Interface Validation Table

Interface Property

Why is this interface this value?

Why do you know that your system design details meet or exceed each property (reference block details as needed)?

outside_system_acpwr:

Vnominal: 120VAC	Standard nominal voltage for AC mains supply in North America.	The power adapter converts this voltage to 5V DC. [11]
Other: NEMA 5-15R	Standard North American AC power plug type.	Ensures compatibility with standard wall outlets. [11]
Inominal: 500mA	Estimated average current drawn during operation.	Based on ESP32 power consumption and servo requirements.

Ipeak: 3A	Accounts for transient inrush current when servos are active.	Ensures the power supply can handle momentary current surges.
-----------	---	---

usrin_GCode_dsig:

(x,y) G-code coordinate range : (0 : 10.25 , 0 : 10.25) inches	This range allows for full reach of the available coordinate plane, currently shown in inches. This can be easily adjusted to accept a wide variety of projects with different units by simply changing the scale.	Python will ensure all G-Code inputs fall within the defined range, an error message will be thrown if this is not true.
G-code speed range: 1-120 mm/s	This range allows us to meet the system requirement of a 10 inch line in 2.5 seconds (102mm/s) with a margin of 18mm/s	Python will provide an error if speed inputs are outside of this range, ensuring all inputs are within this range.

Environment_environment_drawing:

Speed: 4 in/s	This is an engineering requirement that we must meet.	We feed code that commands the system to draw a straight line. By timing the time taken to draw a line of said distance, we can calculate the speed at which the system can draw. It will be able to draw at 4 in/s
Accuracy: +/- 0.25 in. in all directions	This is an engineering requirement that we must meet.	After the system has drawn the line, we can draw from point to point how a perfectly straight line should look. By using this hand-drawn line as a reference, we can measure how far off the robot-drawn line is. It will not be off by more than 0.25 inches in any direction.

Engineering Requirements

1. The system should be fast.
 - a. The system must draw with a speed of 4 inches per second.
2. The system must be accurate.
 - a. The system must draw a 10 inch straight line +/- .25 inch in all directions.
3. The system needs to be inexpensive and manageable to manufacture.
 - a. The system will use a SCARA topology, with two rotating joints to control arm actuation.
4. The system must have a commonly known interface.
 - a. The system input will be G-Code commands. These commands must be made available within the Python or MATLAB GUI.
5. The system must have an emergency stop.
 - a. The system will stop moving in less than 1 second after user input and then return to normal operation when the emergency stop input is reset.
6. The System must be able to change tools.
 - a. The system will provide the user a mechanism to manually swap between various writing utensils within 30 seconds of stopping the system.

Verification Process

1. System Setup
 - a. Place ESP32 microcontroller board and PCA9685 servo driver board in dedicated slots on the PCB board.
 - b. Attach servos to PCA9685 ports 0-3.
 - i. Port 0: Shoulder Servo.
 - ii. Port 1: Elbow Servo.
 - iii. Port 2: Pen-Lifting Servo.
 - c. Ensure the emergency stop is ON.
 - d. Flash code onto ESP32 using Arduino version 1.8.
 - i. Hold the boot button while flashing.
 - e. Compile G-Code
 - i. Open VS code Python environment and press start/compile.
 - ii. Either paste G-Code into the gcode.txt file and enter "n", or enter "y" if you'd like to input G-Code to the terminal.
 - iii. After inputting G-Code following the guide, enter "done" to compile G-Code.
 - iv. Copy the "angles_output.txt" file into the "angles.txt" SPIFFS file in the Arduino code directory.
 - f. Flash SPIFFS data (angles.txt file) onto the ESP32
 - i. Under Tools press "ESP32 Sketch Data Upload".
 - ii. Hold the boot button while flashing.
 - g. Disconnect the USB cable from the ESP32.

- h. Plug the AC-DC converter into the wall, and plug the barrel jack into the PCB barrel jack port.
 - i. Place the drawing surface (paper) into the tray and use tape to hold paper in place if desired.
 - j. When ready, set the emergency stop switch to OFF. (The G-code will now execute)
- 2. Speed
 - a. Apply code that draws a straight, 10-inch line on a piece of paper.
 - b. Using a camera (at least 10 FPS), record the line being drawn.
 - i. Measure the line using a known length (ruler) to verify it is 10 inches.
 - ii. Watch the replay, noting the time it took to draw the 10 inch line was less than 2.5 seconds.
- 3. Accuracy
 - a. Apply code that draws a straight, 10-inch line on a piece of paper.
 - b. Using a ruler, draw a line from endpoint to endpoint.
 - i. We will use this as a reference line of where the line should be located.
 - c. Activate the ESP-32/Robot and allow it to execute the G-Code.
 - i. Again using a known length, verify that the maximum distance from the known line is no more than 0.25-inch in all directions.
- 4. Manufacturing Time and Cost
 - a. The critical path for our project manufacturing was PCB fabrication and delivery time (10 days), everything else was delivered significantly faster than this.
 - i. The manufacturing of other parts, including assembly, 3D-printing, and machining the brass sleeve, could be completed in less than 24 hours.
 - 1. 3D-printing: ~20 hours using a single printer.
 - 2. Brass sleeve machining: ~30 minutes.
 - 3. Assembly ~1 hour.
 - b. The cost of our project was \$108, which is less than the \$200 budget we set.
- 5. G-Code Input
 - a. Apply code that draws anything the user wishes.
 - b. Activate the ESP-32 G-Code
 - i. Validate that the robot executed the specified G-Code by comparing G-Code input to the system drawing output.
- 6. Emergency Stop
 - a. Apply G-code to ESP-32 (Any program works)
 - b. Activate the ESP-32 G-Code
 - i. Record the execution of the G-Code using a camera (at least 10 FPS).
 - ii. Before the G-Code is finished executing, activate the emergency stop switch to ON.
 - iii. After the system has come to a complete stop, switch the emergency stop back to OFF.
 - c. Review the recording of the G-Code execution, ensuring the system came to a stop within 1 second of the emergency stop switch being set to ON and that the G-Code resumed after being switched back to OFF.

7. Tool Change

- a. Using a stopwatch or camera, begin recording.
- b. Swap the tool in the brass-sleeve end effector.
 - i. Remove the Allen wrench from the frame (located below the belt)
 - ii. Lift the brass sleeve out of the end of the elbow arm.
 - iii. Take note of how far the current writing utensil extrudes from the brass sleeve.
 - iv. Using the allen wrench loosen the screw in the side of the brass sleeve (the writing utensil should fall out).
 - v. Place the new writing utensil in the brass sleeve at a similar extrusion length as the original writing utensil.
- c. Review the recording to ensure that this process was completed within 30 seconds.

Life Cycle Assessment

1. What were the main events? Identify a few (at least 2) key turning points from the past few months.
 - a. Frame redesign. Our initial design was not structurally sound and was bending due to the weight of the motors and torque from the belt system.
 - i. To fix this, we redesigned our frame using an increased shoulder-arm size, removing the belt which was causing torque and using a new servo mounting system.
 - b. Calibration. After our system was put together, we tested drawing a straight line and noticed that it was not meeting the accuracy engineering requirement. We recognized that this was due to the motors not moving to exactly the angle we set. It was always off by a few degrees.
 - i. To combat this problem, we tested each angle and found the difference for each to see what value needed to be imputed to each motor to get an accurate movement.
2. What were the most important challenges you faced? How did you overcome them or reduce their overall impact on your project?
 - a. The main challenge we faced was adjusting the motor to meet the engineering accuracy requirement. It required coming up with code and ideas on how to alleviate the issue.
3. If another team took over this project right now, what is the most important information they would need to move forward?
 - a. The most important piece of information you should know before beginning this project is that the servos are not the most accurate. If you are to follow our idea and use servo motors, we would recommend creating a more accurate calibration system than we did. The process can be similar to the one done by [Jeremy Lindsay](#) [14].
4. What advice would you give that new team starting from now?

Got feedback on this document? [Email us!](#)

- a. We would recommend using stepper motors as they are more accurate in small movements. This change would have helped to improve accuracy.
5. What advice would you give yourself if you could relay the information at the start of the term?
 - a. Start early on the PCB designing and make sure to order parts and prototype early. We began designing the PCB midway through the schedule, and due to complications and shipping time, we were not left with a generous amount of time to test different designs and optimize. By starting earlier, we can go through different iterations and use the PCB as a base for the rest of the project.
6. What would you like to say about the strengths of your team? Every team is made of unique individuals who contributed in some way to the project execution. Take a moment to thank your team members with specific examples of how they influenced this project. Use the template below.
 - a. From Gabe, to Dylan and Alex. Thank you Alex for designing our PCB in this project and implementing a super convenient wall-DC converter. Thank you Dylan for being a universal team player picking up slack whenever needed.
 - b. From Dylan, to Gabe and Alex: Thank you for all of your contributions to this project, pleasure working with you. Thank you, Alex and Gabe, for designing a multitude of key components for this project. The different perspectives you provided were enlightening.
 - c. From Alex, to Gabe and Dylan: Thank you, Gabe, for designing and showcasing an elite knowledge of CAD in our arm design. Thank you Dylan, for being available to help whenever we were stuck on an issue.

Artifacts

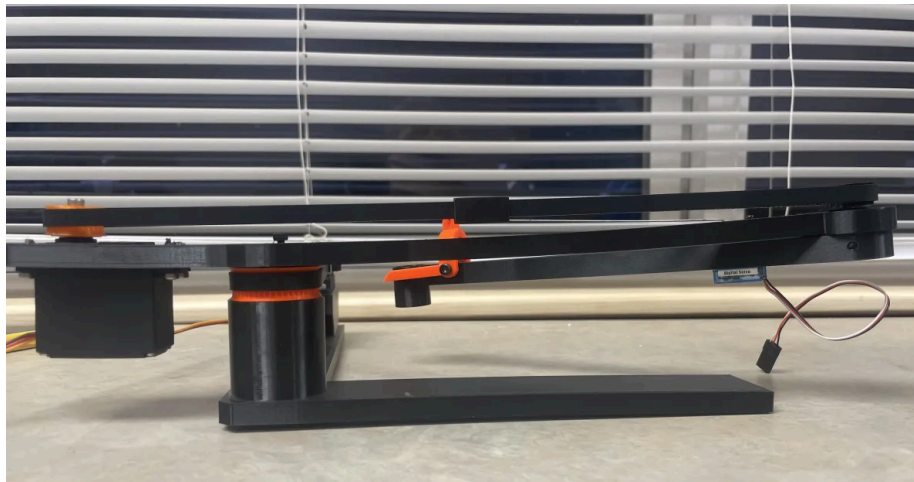


Figure #23: Initial frame design, thinness of parts caused bending and torque issues.

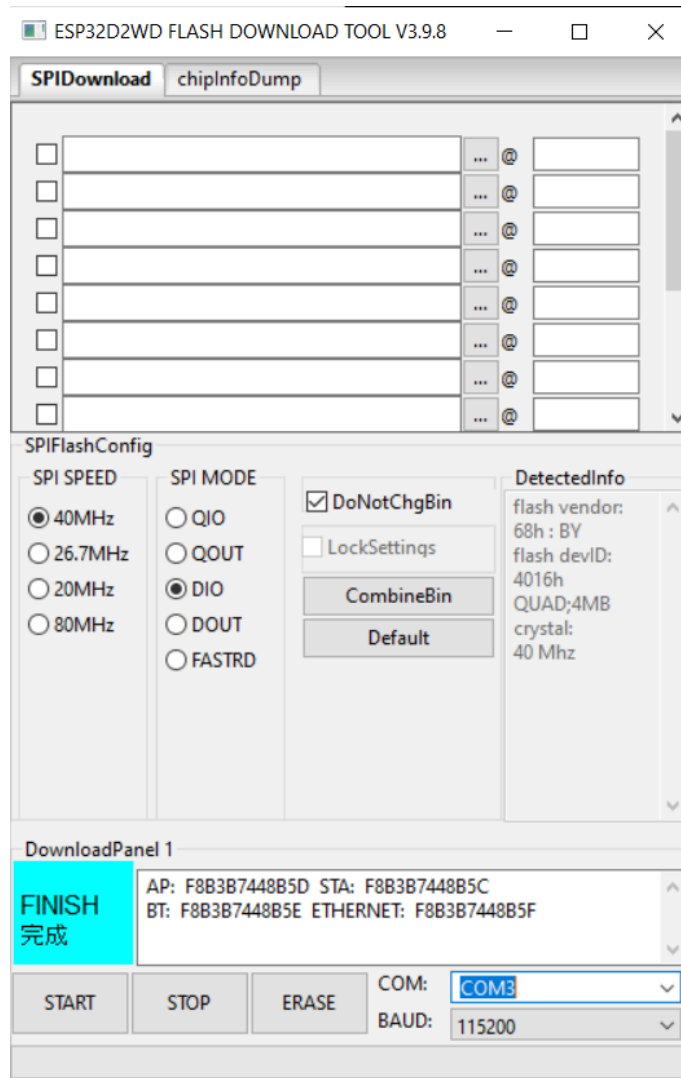


Figure #24: ESP32 Details

A1 Expected

	A	B	C	D	E	F	G	H
1	Expected	Actual	Difference	Using Input	Col D + 1 deg	Difference input	Average	Differene of diff
2	180	180	0	179	180	0	0	0
3	170	171.5	1.5	169	170	0	0.75	-1.5
4	160	162	2	160	161	1	1.5	-1
5	150	153.5	3.5	152	153	3	3.25	-0.5
6	140	140.5	0.5	140.5	141.5	1.5	1	1
7	130	130.5	0.5	129	130	0	0.25	-0.5
8	120	119	-1	119	120	0	-0.5	1
9	110	108	-2	109.5	110.5	0.5	-0.75	2.5
10	100	99.5	-0.5	99	100	0	-0.25	0.5
11	90	89.5	-0.5	90	91	1	0.25	1.5
12	80	78	-2	79	80	0	-1	2
13	70	69	-1	69.5	70.5	0.5	-0.25	1.5
14	60	58	-2	58	59	-1	-1.5	1
15	50	47.5	-2.5	48	49	-1	-1.75	1.5
16	40	35	-5	35	36	-4	-4.5	1
17	30	27	-3	27	28	-2	-2.5	1
18	20	19	-1	19	20	0	-0.5	1
19	10	9.5	-0.5	9	10	0	-0.25	0.5
20	0	0	0	0.5	1.5	1.5	0.75	1.5

Figure #25: Servo Calibration Process

The important information from this figure is our flash size which is QUAD;4MB, telling us how much storage we have for our file input.



Figure #26: Pen Lifting Servo Dimensions and Name (GH-S37D) [10]

References

- [1] "MG996R servo motor," Components101, [Online]. Available:
<https://components101.com/motors/mg996r-servo-motor-datasheet>
- [2] [3DSourced 3DSourced is a leading authority in the field of 3D printing and scanning and 3DSourced is a leading authority in the field of 3D printing and scanning, "How strong are 3D printed parts? (PLA, ABS, Petg & More)," 3DSourced, [Online]. Available:
<https://www.3dsourced.com/guides/how-strong-are-3d-printed-parts-pla-abs-petg/>
- [3] Espressif Systems, "ESP32-WROOM-32 Datasheet," [Online]. Available:
https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf.
- [4] Electronics Hub, "ESP32 Pinout: A Complete Guide," [Online]. Available:
<https://www.electronicshub.org/esp32-pinout/>.
- [5] SnapEDA, "ESP32-WROOM-32 Component Details," [Online]. Available:
<https://www.snapeda.com/>.
- [6] NXP Semiconductors, "PCA9685: 16-channel, 12-bit PWM Fm+ I2C-bus LED controller datasheet," [Online]. Available:
<https://www.digikey.com/htmldatasheets/production/1640697/0/0/1/PCA9685-Datasheet.pdf>.
- [7] J. Kim, "Analysis and Optimization of DC Supply Range for the ESP32 Development Board," [Online].
- [8] Adafruit PCA9685 16-channel servo driver, [Online]. Available:
<https://cdn-learn.adafruit.com/downloads/pdf/16-channel-pwm-servo-driver.pdf>
- [9] J. Tervit, "ESP32: I2C Scanner (Arduino)," Random Nerd Tutorials, [Online]. Available:
<https://randomnerdtutorials.com/esp32-i2c-scanner-arduino/>
- [10] "System design and project artifacts," Google Docs, [Online]. Available:
<https://docs.google.com/document/d/1LFXEKM5IRDq9PwHfho0SXh2JqFYM05kgtKQrL1qkSd0/edit?tab=t.0>
- [11] "Amazon.com: Shanghj 2PCS GH-S37D 3.7G Mini Digital Servo Super Light 3.6-4.8V micro coreless servo for control aircraft flight direction RC Airplane Car Boat, blue : Toys & games," [Online]. Available:
<https://www.amazon.com/ShangHJ-3-6-4-8V-Coreless-Aircraft-Direction/dp/B096KBWFKC>
- [12] Roboticsqut, "Inverse kinematics for a 2-joint robot arm using geometry," YouTube, [Online]. Available:
https://www.youtube.com/watch?v=IKOGwoJ2HLk&embeds_referring_euri=https%3A%2F%2Frobotacademy.net.au%2F&source_ve_path=MjM4NTE

Gabriel Boyd 934-428-158, Dylan Sutardjo 934-434-234, Alexander Prosnik 934-431-515
Team #3

[13] "Download," Tools | Espressif Systems, [Online]. Available:
<https://www.espressif.com/en/support/download/other-tools>

[14] J. Lindsay, "Calibrating my servos," Hackster.io,
<https://www.hackster.io/jeremy-lindsay/calibrating-my-servos-fa27ce>